

QR BASED - INTELLIGENT HOSPITAL ASSETTRACKING AND MANAGEMENT SYSTEM



A Project Report

Submitted by

AMBIKA B	611223114003
ARAVINTH R	611223114006
BHUVANAN M	611223114013
DHARANESHWARAN B	611223114020

In partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

MECHANICAL ENGINEERING

KNOWLEDGE INSTITUTE OF TECHNOLOGY, SALEM

(An Autonomous Institution)

ANNA UNIVERSITY :: CHENNAI 600 025

AUG 2026

BONAFIDE CERTIFICATE

Certified that this project report **“INTELLIGENT HOSPITAL ASSET TRACKING AND MANAGEMENT SYSTEM”** is the Bonafide work of **“AMBIKA B, ARAVINTH R, BHUVANAN M, and DHARANESHWARAN B”** who carried out the project work under my supervision.

SIGNATURE

Dr. K.S. PRABHAKARAN, M.E., Ph.D.

HEAD OF THE DEPARTMENT

Associate Professor

Department of Mechanical Engineering

Knowledge Institute of Technology

Salem – 637 504.

SIGNATURE

Mr. A. KAMALAKKANNAN, M.E., (Ph.D),

SUPERVISOR

Assistant Professor

Department of Mechanical Engineering

Knowledge Institute of Technology

Salem – 637 504.

ACKNOWLEDGEMENT

We thank our beloved Executive Chairman and President **Dr.PSS.SRINIVASAN, B.E., M.Tech., Ph.D.**, and all the members of Knowledge Institute of Technology Trust at this high time for providing us with excess of facilities to complete the project successfully.

My heartfelt gratitude to **Dr. K. VISAGAVEL, M.E., Ph.D., FIE**, Principal of KIOT, for his invaluable guidance and encouragement throughout the journey.

I am very much obliged to **Dr. K.S. PRABHAKARAN, M.E., Ph.D.**, Head of the Department, Mechanical Engineering, for his valuable insights that significantly enhanced the quality of my project.

I am humbled and thankful **Mr. S. SHANMUGAM, M.E.**, Professor of Practice, Department of Mechanical Engineering for Initiating the Institute To Industry (I2I) Course and Providing Valuable Industry Exposure . His guidance and motivation greatly enhanced our learning and contributed to the Successful completion of the project.

A sincere thanks to my Project Work Supervisor **Mr.A. KAMALAKKANNAN, M.E.,(Ph.D.)**, Department of Mechanical Engineering, Department of Mechanical Engineering, for his exceptional guidance and unwavering support throughout the project's duration.

I also wish to acknowledge the entire teaching faculty, supporting staffs of the Mechanical Engineering Department, and my fellow students for their assistance and cooperation, which played a vital role in bringing the project to fruition.

I am forever thankful to all who contributed for the successful completion of the project

ABSTRACT

Hospitals depend on a large number of reusable movable assets such as wheelchairs, stretchers, oxygen cylinders, infusion pumps, portable monitors and diagnostic equipment that move continuously between wards, ICUs, operation theatres, stores and diagnostic rooms. When this movement is tracked manually through notebooks, spreadsheets or verbal communication, the recorded information quickly becomes outdated, leading to delays in locating assets, unnoticed shortages, delayed returns and unreported damage. This project presents the design and development of Asset Track, a low-cost, browser-based Intelligent Hospital Asset Tracking and Management System that uses QR-code scanning to identify each reusable asset, record the employee currently using it, confirm its latest scanned location, and automatically update its availability status. The system provides three separate role-based portals – Nurse & Staff, Office and Management – each with permissions suited to their responsibilities, ranging from taking, returning and transferring assets to monitoring dashboards, reports, low-availability warnings, maintenance history, internal messaging and emergency alerts. The application is developed using a Python Flask backend, an SQLite database and an HTML, CSS and JavaScript frontend, and is deployed as a live HTTPS web application hosted on PythonAnywhere so that it can be accessed from mobile phones, tablets, laptops and desktop computers without any specialised tracking hardware. The system was tested for login and role validation, asset take/return/transfer cycles, duplicate-action prevention, damage and maintenance reporting, low-availability and delayed-return detection, messaging and emergency workflows. The results confirm that the QR-based confirmation approach reliably identifies the current holder and latest location of every asset, reduces asset search time, exposes shortages before they disrupt patient care, and gives Office and Management staff a single, traceable view of hospital asset activity. The study demonstrates that an intelligent, software-driven asset tracking approach can significantly improve the efficiency, accountability and transparency of hospital asset management without the cost of RFID, BLE or GPS-based hardware infrastructure.

Keywords: Hospital Asset Tracking, QR Code Scanning, Flask Web Application, SQLite Database, Role-Based Access Control, Asset Lifecycle Management, Real-Time Dashboard, Healthcare Management System.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF SYMBOLS	ix
	ABBREVIATIONS	x
	NOMENCLATURE	xi
1	INTRODUCTION	1
1.1	Problem Statement	1
1.2	Objectives and Scope	2
1.3	Methodology	
2	LITERATURE SURVEY	6
3	SYSTEM REQUIREMENTS, COMPONENTS AND SPECIFICATION	8
3.1	Hardware Requirements	8
3.2	Software Requirements and Technology Stack	9
3.3	Components Description	
4	SYSTEM ARCHITECTURE AND WORKING PRINCIPLE	11
4.1	System Architecture	11
4.2	System Users and Permissions	12

4.3	Complete System Working Process	13
4.4	Automatic Availability and Return Logic	15
4.5	Messaging, Emergency and Maintenance	16
5	SYSTEM DEVELOPMENT AND IMPLEMENTATION	17
5.1	Module-by-Module Development Process	17
5.2	Database Design	21
5.3	Project Folder and File Structure	22
5.4	Request Flow and User Interface	24
6	RESULT AND DISCUSSION	26
6.1	Testing and Demonstration Procedure	26
6.2	Results and Discussion	33
7	SCOPE OF FUTURE WORK	34
7.1	Advantages	34
7.2	Limitations	35
7.3	Applications	
8	APPENDICES	36
	Appendix – 1 Login Credentials	36
	Appendix – 2 Database Schema Specification	
	Appendix – 3 Pseudo Code for Key Functions	37
	Appendix – 4 System Development and Coding Checklist	39
9	CONCLUSION	40
	REFERENCES	41

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 1.2	Scope of project	2
Table 3.1	Hardware requirements for QR-based operation	8
Table 3.2	Software / technology stack summary	9
Table 4.1	System architecture layers and components	12
Table 4.2	System users, portals and permitted actions	
Table 4.4	Availability and return-request states	15
Table 5.1	Main database tables and purpose	20
Table 5.2	Project folder and file explanation	23
Table 6.1	Functional test cases and expected results	26
Table 7.1	Advantages, limitations and future improvements	34

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 1.3	Methodology	3
Figure 4.1	Flask – SQLite system architecture of Asset Track	11
Figure 4.3	Normal asset lifecycle and exception paths	13
Figure 5.1	Module-by-module development process	17
Figure 5.3	Completed project folder used for development and deployment	22
Figure 5.4	How one asset action moves through the code, from browser to database and back	24
Figure 6.1.2	Asset track portal selection page	27
Figure 6.1.3	Nurse & staff login page	28
Figure 6.1.4	Staff dash board	
Figure 6.1.5	Asset QR code scanning	29
Figure 6.1.6	Current asset overview	
Figure 6.1.7	Hospital assets	30
Figure 6.1.8	Asset QR codes	
Figure 6.1.9	Create new asset	31
Figure 6.1.10	Send Message	
Figure 6.1.11	Hospital asset analytics	32

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

1. LIST OF SYMBOLS

SYMBOL	FULL FORM
QR	Quick Response Code
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
SQL	Structured Query Language
DB	Database
ID	Identification
ICU	Intensive Care Unit
UI	User Interface
API	Application Programming Interface
WSGI	Web Server Gateway Interface
CRUD	Create, Read, Update, Delete
RFID	Radio Frequency Identification
BLE	Bluetooth Low Energy
GPS	Global Positioning System
HIS	Hospital Information System

2. ABBREVIATIONS

ABBREVIATION	FULL FORM
QR	Quick Response (Code)
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
SQL	Structured Query Language
DB	Database
ID	Identification
ICU	Intensive Care Unit
UI	User Interface
API	Application Programming Interface
WSGI	Web Server Gateway Interface
CRUD	Create, Read, Update, Delete
RFID	Radio Frequency Identification
BLE	Bluetooth Low Energy
GPS	Global Positioning System
HIS	Hospital Information System

3. NOMENCLATURE

TERM	DEFINITION
Asset	A reusable hospital item (wheelchair, monitor, pump, etc.) individually identified with a unique ID and QR code.
Take	The action of assigning an available asset to an employee at a scanned location.
Return	The action of releasing an in-use asset back to Available status at an authorised location.
Transfer	Movement of an in-use asset to another location or responsible employee without releasing it.
Latest confirmed location	The last location recorded through a QR scan or transaction, as opposed to continuous live tracking.
Low availability	A state where the available count of an asset type falls below its configured minimum quantity.
Delayed status	A state where a promised return time has passed without the asset being returned.
Role-based access	Restriction of system functions based on the logged-in user's assigned portal (Staff, Office or Management).

CHAPTER 1

INTRODUCTION

Hospitals rely on a large number of reusables, movable assets such as wheelchairs, stretchers, oxygen cylinders, infusion pumps, portable monitors, emergency equipment and diagnostic devices. These assets move continuously between wards, ICU rooms, operation theatres, diagnostic rooms, stores, entrances and ambulance areas as patient care activity changes throughout the day. Timely availability of the correct asset at the correct location is directly linked to the quality and speed of patient care.

When this movement is recorded only through notebooks, spreadsheets, verbal handovers or informal communication between staff, the recorded information becomes outdated within a short time. Staff may know that an asset exists somewhere in the hospital but may not know who currently holds it, where it was last moved, whether it is available for use, whether its return is overdue, or whether it has been reported as damaged. This uncertainty leads to duplicate purchases, delayed treatment, wasted staff time searching for equipment, and an inability of management to make informed decisions about asset utilisation.

The rapid growth of low-cost smartphones, browser-based camera access and lightweight web frameworks has made it possible to build practical, software-only asset tracking solutions without investing in expensive RFID readers, BLE beacons or GPS hardware. A unique QR code fixed to each physical asset, combined with a simple web application, is sufficient to identify an asset, confirm its latest location and record who is responsible for it at any given time.

This project presents Asset Track, an Intelligent Hospital Asset Tracking and Management System that uses QR-code scanning, a Python Flask backend and an SQLite database to give Nurse & Staff, Office and Management users a shared, browser-based view of hospital asset activity. The system is designed to be low-cost, easy to deploy, and usable on any modern mobile phone, tablet, laptop or desktop computer with a camera and a web browser.

1.1 Problem Statement

Problem statement:

A hospital needs a low-cost, common system to identify reusable assets, record the employee using each asset, confirm the latest location, control availability, request timely returns, report damage, preserve movement history and provide clear management reports.

Manual or spreadsheet-based tracking cannot answer these questions in real time. As the number of assets and departments grows, the effort required to keep such records accurate becomes unmanageable, and the resulting blind spots directly affect the availability of critical equipment during emergencies.

1.2 Objectives and Scope

Main objective: to develop a secure, browser-based asset tracking and management system that works on mobile phones, tablets, laptops and desktop computers without requiring specialised tracking hardware.

- Provide separate login portals for Nurse & Staff, Office and Management users, each with permissions suited to their responsibilities.
- Give every reusable asset a unique identification number and QR code.
- Record asset status, condition, current holder, latest confirmed location and transaction time for every action.
- Update dashboards and availability count automatically after every action.
- Detect low availability of an asset type and support return requests for assets already in use.
- Support transfer between locations, damage reporting, maintenance tracking, internal messaging, emergency alerts and management reports.
- Allow Office users to add employees, departments, locations, asset types and individual assets without changing the application source code.

Aspect	Description
Included	Reusable movable hospital assets, user assignment, QR-based location confirmation, status changes, return management, communication, maintenance and reports.
Not included	Consumable stock such as medicines, gloves, masks and syringes, which require inventory quantity management rather than individual-asset tracking.
Tracking meaning	The system stores the latest confirmed location after a scan or transaction. It does not continuously show the physical asset position like GPS.
Supported devices	Modern browsers on mobile phones, tablets, laptops and desktop computers. Camera-based scanning needs HTTPS and browser camera permission.

Table 1.2 Scope of the project

1.3 Methodology

The project was developed using a systematic, module-by-module methodology that moves from requirement understanding to a fully deployed, tested web application. The overall methodology consists of the following stages:



Figure 1.3 Methodology

1.3.1 Requirement Study

The first stage of the project focused on understanding the difficulties involved in managing movable assets within a hospital. The commonly used assets, such as wheelchairs, stretchers, portable monitors and other reusable equipment, were identified along with the departments in which they are normally used. The requirements of different users, including Staff, Office and Management, were studied to understand the actions and information required by each group. The need for asset identification, location confirmation, availability monitoring, return management and reporting was also analysed. Based on this study, a QR-code-based approach was selected to provide a practical method of tracking the latest confirmed location without depending on costly GPS or RFID infrastructure.

1.3.2 Workflow and Role Design

After identifying the requirements, the complete workflow of the proposed system was planned according to the responsibilities of each user. Separate activities were defined for Staff, Office and Management so that each user could perform only the operations related to their role. The different asset conditions, such as Available, In Use, Return Requested, Delayed and Maintenance, were mapped before the implementation started. The movement of an asset from

one state to another was carefully defined. This helped prevent incorrect operations, such as assigning an asset that was already being used by another employee.

1.2.3 Database Design

A structured database was designed to store both the current condition of assets and their previous activities. SQLite was selected as the database because it is simple to configure and suitable for the proposed application. Separate tables were planned for users, departments, locations, asset categories, asset types, individual assets, transactions, return requests, maintenance records, messages and emergency events. Relationships between these tables were established using unique IDs and foreign keys. The database structure was designed so that the current status of an asset could be accessed quickly while its complete movement history remained available for future reference and reporting.

1.2.4 Flask Backend Development

The backend of the application was developed using Python Flask. The main application file, app.py, manages the communication between the user interface and the database. Different routes were created for login, asset operations, QR scanning, dashboards, reports and other system functions. Before performing an operation, the backend verifies the user's session and role and checks whether the requested asset action is valid. Database records are then updated according to the operation performed. This approach ensures that the important business rules of the asset management system are handled centrally and consistently.

1.2.5 Frontend Development

A responsive frontend was developed so that the application could be used on different devices available within a hospital environment. HTML templates were created for login pages, dashboards, asset operations, scanning screens, reports and other functions. CSS was used to maintain a clear and adaptable interface for mobile phones, tablets and computers. JavaScript was incorporated for interactive elements, form validation, camera access and QR-code scanning. The interface was designed with simple navigation so that Staff can perform quick asset-related operations while Office and Management users can conveniently monitor information from larger screens.

1.2.6 QR and Camera Integration

QR-code integration was implemented as one of the main features of the proposed system. A unique QR value was assigned to each individual asset and authorised location. When a Staff user scans an asset QR code using a mobile device, the browser camera reads the encoded information and sends it to the Flask application. The system then identifies the corresponding asset and displays its current status, condition and latest confirmed location. Location scanning can also be used during asset movement to record where the equipment was taken or returned. This provides a simple tracking method without requiring dedicated scanning equipment.

1.2.7 Authentication and Permissions

A role-based authentication mechanism was implemented to protect the system from unauthorised access. Separate access levels were provided for Staff, Office and Management users. During login, the system verifies the entered credentials and checks whether the selected portal matches the user's assigned role. Passwords are stored using secure hashing rather than plain text. Session management is used to maintain the authenticated user's identity while

accessing protected pages. Logout and password recovery facilities were also considered to improve account management and system security.

1.2.8 Reports and Alerts

Once the basic asset Take and Return operations were functioning correctly, additional monitoring and communication features were integrated. The system displays asset availability counts, recent transactions and relevant status information through dashboards. A minimum availability level can be configured for different asset types, allowing the system to identify shortages when the available quantity falls below the required level. Return-request and delayed-return mechanisms were also incorporated to support timely recovery of assets. In addition, maintenance records, internal messages and emergency alerts were included to provide a broader operational management facility.

1.2.9 Local Testing

Before deployment, the complete application was tested in a local development environment. A Python virtual environment was used to maintain the required software dependencies separately from the main system. Different user accounts and asset conditions were used to verify the application workflow. Tests were carried out for login validation, asset taking, returning, transferring, damage reporting, maintenance, availability warnings, delayed returns, messaging and emergency alerts. Mobile camera testing was also performed to verify QR scanning under secure browser access. Invalid operations were intentionally attempted to confirm that the system correctly rejects actions that do not follow the defined workflow.

1.2.10 Deployment

After completing the local development and testing stages, the application was prepared for online deployment. The Flask application, database, templates, static files and required Python packages were configured for the hosting environment. PythonAnywhere was used to make the application accessible through a live HTTPS connection. The WSGI configuration was connected to the Flask application so that the server could handle incoming browser requests. The deployed system was then checked again to ensure that login, database operations, QR scanning and the major user functions continued to operate correctly in the online environment.

1.2.11 Documentation and Maintenance

The final stage involved preparing supporting documentation and maintaining the developed application. A README file was maintained to describe the installation, execution and general usage procedure. Update notes were used to record important modifications and corrections made during development. A password-recovery utility was included for account management, while database backups were maintained to reduce the risk of losing operational information. These practices make the application easier to maintain, update and reproduce when the system is extended or transferred to another environment.

CHAPTER 2

LITERATURE SURVEY

- **Impact of Tracking Technology Adoption in Hospitals (2022)**

This study examined the adoption of tracking technologies such as barcodes and RFID in hospitals. It reported that tracking systems improve equipment identification, information sharing and operational efficiency, supporting the need for a digital hospital asset tracking system.

- **RFID Deployment for Hospital Asset Tracking (2022)**

The study investigated the use of an RFID-based real-time location system for tracking IV pumps in a medical centre. It showed that automated tracking improves the management and availability of movable medical equipment, but RFID deployment requires additional infrastructure and organizational support.

- **Online Asset Monitoring Using QR Code Technology (2023)**

Moonsrikaew, Sitawan and Chatsermsak developed an online asset monitoring system using QR codes, smartphones and a web application. The study demonstrated that QR-based identification can provide centralized asset records, movement monitoring and reporting without requiring specialized tracking hardware.

- **QR-Based Asset Lifecycle Management System (2024)**

Azriani and Frendiana designed a web-based asset lifecycle management system using QR Code technology. Their system covered asset planning, acquisition, operation, maintenance and disposal, showing that QR-based systems can improve the accuracy and transparency of asset management throughout the complete asset lifecycle.

- **Barcode-Based Medical Asset Management in Hospitals (2025)**

Azzahra and Suakanto developed a medical asset management system for hospitals with centralized data entry, QR code generation, search filters and a real-time dashboard. The system achieved successful functional testing and high user satisfaction, supporting the use of QR-based identification for hospital asset monitoring.

- **Web-Based QR Asset Tracking with Role-Based Access Control (2026)**

Sitanggang, Saputra, Hidayati and Saragih developed a web-based asset tracking system using QR codes and role-based access control. The system provided unique asset identification, location mapping, audit trails and separate access permissions, demonstrating the importance of accountability and controlled user access in asset management systems.

- **Secure Role-Based Access Control for Tracking Applications (2023)**

Recent tracking system research has integrated Role-Based Access Control with secure tracking architectures to protect asset information and restrict unauthorized access. This supports the use of separate Staff, Office and Management portals in the proposed hospital asset tracking system.

- **Recent Secure QR and Role-Based Healthcare Tracking Systems (2026)**

Recent healthcare research has shown that QR-based retrieval combined with role-based access control can improve usability, accountability and secure information access. This approach supports the proposed system's QR-based asset identification and controlled access to hospital asset records.

- **Patel and Shah (2018)**

The authors evaluated QR-code-based inventory and asset management applications in small and medium enterprises and found that browser-based QR scanning, without dedicated scanner hardware, was sufficient for accurate identification when paired with a reliable backend database.

- **Ahsan and Shah (2019)**

This review of healthcare information systems highlighted those web-based applications built on lightweight frameworks (such as Flask and Django) with relational databases provide a practical, low-cost route to digitising hospital operations compared to large, vendor-supplied hospital information systems.

- **Rao and Nguyen (2021)**

The authors studied role-based access control in web-based hospital management systems and concluded that separating user portals by responsibility (clinical staff, administrative staff and management) reduces accidental data modification and improves accountability for recorded actions.

The literature confirms two consistent findings that directly shaped this project. First, automated identification – whether by RFID or QR code – significantly reduces the time staff spend searching for shared equipment and improves the accuracy of availability information. Second, a browser-based, database-backed system built on an inexpensive identification method such as QR scanning can deliver most of the operational benefit of more expensive real-time tracking hardware, which supports the choice of a Flask and SQLite-based QR system for this project.

CHAPTER 3

SYSTEM REQUIREMENTS, COMPONENTS AND SPECIFICATION

3.1 Hardware Requirements

Asset Track is a software-based hospital asset management system designed to function without the need for specialised tracking equipment such as RFID readers, GPS modules, BLE devices, or dedicated tracking terminals. Instead, the system makes use of commonly available digital devices that are already used in hospital environments. This approach helps reduce the initial infrastructure cost and makes the system easier to implement and maintain. Staff members can access the system through smartphones or tablets equipped with cameras for scanning QR codes attached to hospital assets and authorised locations. Office and management personnel can use laptops or desktop computers to manage asset records, monitor availability, track movements, and generate reports. Therefore, AssetTrack provides a simple, flexible, and cost-effective hardware setup while still supporting efficient asset identification and movement tracking.

S.No	Hardware	Purpose
1	Mobile phone / tablet with rear camera	QR code scanning by Nurse & Staff during Take, Return, Transfer and Damage actions
2	Laptop / desktop computer	Office and Management dashboards, master-data entry and report viewing
3	QR code labels (printed, laminated)	Fixed to each physical asset and to authorised locations for scanning
4	Label / regular printer	Printing QR code labels for assets and locations
5	Internet-connected router / mobile data	Access to the live HTTPS web application from any department
6	Application server (cloud-hosted)	Runs the Flask application and SQLite database centrally

Table 3.1 Hardware requirements for QR-based operation

3.2 Software Requirements and Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	Responsive pages, forms, dashboards and the QR camera interface
Backend	Python 3 with Flask	Routes, authentication, validation and business logic
Database	SQLite (meditrack.db)	Permanent storage of project data and transaction history
Security	Werkzeug password hashing, Flask sessions	Protected passwords and role-based access control
Hosting	PythonAnywhere with HTTPS	Public deployment and secure browser camera access
Local runtime	.venv virtual environment, Windows batch files	Repeatable local execution and mobile camera testing

Table 3.2 Software / technology stack summary

3.3 Components Description

3.3.1 Python Flask

Flask is a lightweight Python web framework used as the application server for Asset Track. It receives every browser request, checks the login session and user role, applies asset business rules such as status validation, reads or writes records in the SQLite database and renders the correct HTML page or JSON response. Flask was selected because it is simple to learn, has a small footprint suitable for low-cost hosting, and integrates directly with Python's standard sqlite3 library without requiring a separate database server.

3.3.2 SQLite Database

SQLite is a self-contained, file-based relational database used to store all operational data of the system in a single file, meditrack.db. It stores users, departments, locations, asset categories, asset types, individual assets, transactions, return requests, maintenance records, messages and emergencies. SQLite was chosen for its zero-configuration setup, reliability for a moderate transaction volume, and ease of backup, since the complete database can be copied as a single file.

3.3.3 HTML, CSS and JavaScript

HTML templates rendered through Flask's Jinja2 templating engine present the login pages, dashboards, forms, scan screens, tables and reports. CSS provides a responsive layout so the

same interface adapts to mobile, tablet and desktop screens. JavaScript handles camera access for QR scanning, client-side validation, dynamic updates and interactive alerts without requiring a full page reload for every action.

3.3.4 QR Code Scanning (Browser Camera API)

QR scanning is implemented using the browser's camera access API together with a JavaScript decoding library. When a user opens the scan page, the script activates the device camera, continuously decodes visible QR patterns, and sends the decoded value to the Flask backend through an API call. This approach avoids the cost of dedicated barcode/QR scanner hardware while remaining fast enough for routine ward use.

3.3.5 Werkzeug Security

Werkzeug, the WSGI utility library bundled with Flask, provides password hashing functions used to store every user's password as a secure hash rather than as readable text. The same library supports Flask's session-cookie handling, which keeps a logged-in user's identity and role available to every subsequent protected request.

3.3.6 PythonAnywhere Hosting

PythonAnywhere is a cloud application hosting platform used to deploy the Flask application on a public HTTPS URL. It runs the WSGI-configured Flask application, serves the static and template files, and keeps the SQLite database file available to the running application, allowing every hospital department to reach the same live system from any browser.

3.3.7 Local Development Environment

A Python virtual environment (.venv) keeps the project's package dependencies isolated and reproducible during development. Windows batch files (START_MEDTRACK.bat and START_SECURE_MOBILE_LINK.bat) were created to start the local server quickly and to expose it securely over HTTPS on the local network for mobile camera testing before deployment.

CHAPTER 4

SYSTEM ARCHITECTURE AND WORKING PRINCIPLE

4.1 System Architecture

Asset Track uses a simple three-layer web architecture consisting of a browser-based user interface, Flask application server, and SQLite database. The system is hosted on a cloud server and accessed securely through HTTPS. Users can access the application using smartphones, tablets, laptops, or desktop computers through separate Nurse & Staff, Office, and Management portals. The user interface layer allows users to perform activities based on their roles, such as scanning QR codes, taking or returning assets, managing records, and viewing reports. The Flask application layer processes user requests and performs authentication, role verification, asset-status validation, and database operations. The database layer stores user details, asset information, locations, transactions, maintenance records, and other system data.

Whenever a user performs an action, the request is processed by the Flask server and, after validation, the required information is updated in the SQLite database. The updated status is then displayed through the web interface. This architecture provides a simple, secure, centralised, and cost-effective approach for managing hospital assets.

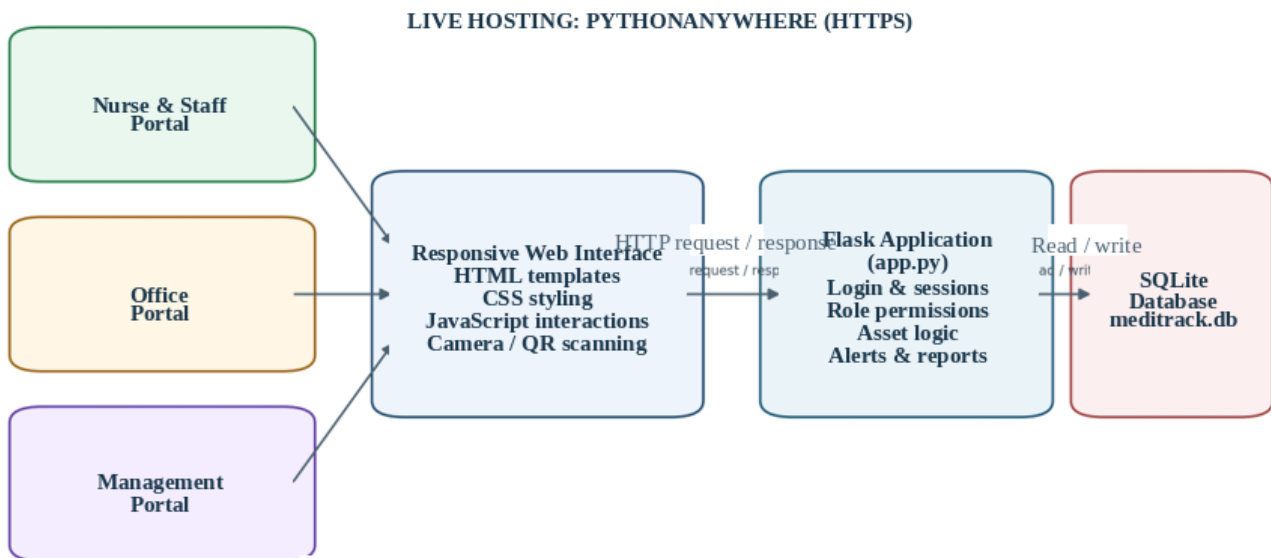


Figure 4.1. Flask – SQLite system architecture of Asset Track

Layer / component	Implementation	Purpose
User interface	HTML templates (Jinja2)	Login pages, dashboards, forms, scan screens, tables and reports
Styling and interaction	CSS and JavaScript (static folder)	Responsive layout, camera scanning, validation, dynamic displays and alerts
Application server	Python Flask (app.py)	Routes, sessions, permissions, business rules, asset actions and report generation
Database	SQLite (meditrack.db)	Stores users, assets, locations, transactions, return requests, maintenance, messages and emergencies
Local environment	.venv and requirements.txt	Keeps required Python packages isolated and reproducible
Cloud hosting	PythonAnywhere	Runs the Flask application through a public HTTPS link

Table 4.1 System architecture layers and components

4.2 System Users and Permissions

Portal	Main responsibilities	Typical permitted actions
Nurse & Staff	Daily asset use and field updates	Login, scan assets, take, return, transfer, view assigned assets, respond to return requests, report damage, read/send messages and acknowledge emergencies
Office	Operational control and master-data management	Create employees, categories, asset types and assets; generate/view QR codes; monitor holders, locations, status, low availability, maintenance, messages and reports
Management	Hospital-wide oversight and decisions	View totals, graphs, recent transactions, asset availability, delayed/maintenance/missing assets, emergency history and consolidated reports

Table 4.2 System users, portals and permitted actions

4.2.1 Login Process

- The user opens the live Asset Track website.
- The user selects the correct portal: Nurse & Staff, Office or Management.
- The user enters the assigned User ID and password.
- The Flask backend checks the account, password and selected role.
- After successful verification, the server creates a login session and opens the correct dashboard.
- Protected pages reject users who do not have the required role.
- The user logs out after completing the work.

Important for real use

Every employee should have an individual account. Shared demonstration accounts should not be used in a real hospital because the system must accurately identify who took, returned, transferred or reported an asset.

4.3 Complete System Working Process

Overall working sequence

Login → identify or scan asset → confirm location/action → validate permission and current status → save transaction in SQLite → update asset state and counts → show the result on Staff, Office and Management dashboards.

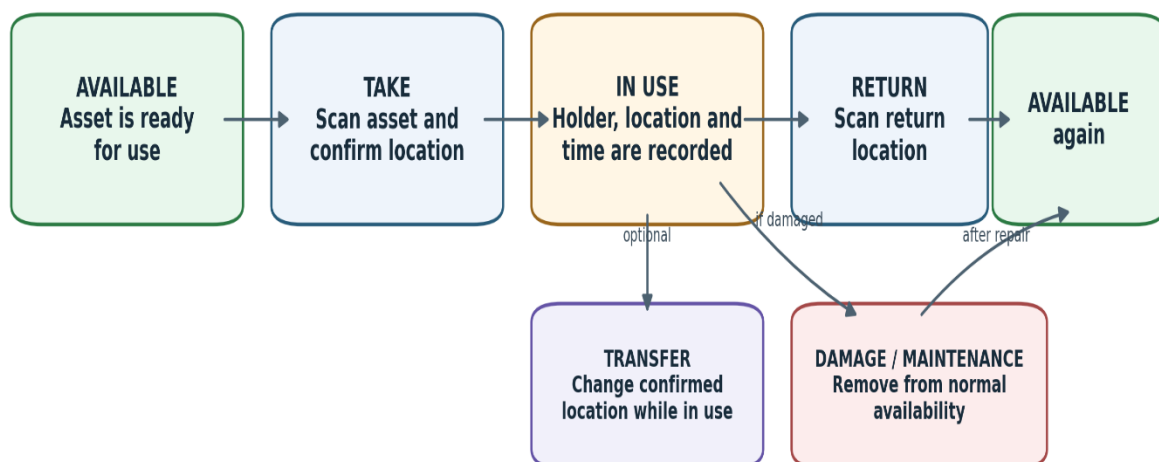


Figure 4.3. Normal asset lifecycle and exception paths

4.3.1 Initial System Setup

- Office users create hospital departments and authorised locations.
- Asset categories and asset types are created, including minimum available quantity and expected return time.
- Each physical asset is added with a unique Asset ID and QR value.
- A QR label is printed and fixed to the corresponding equipment.
- Location QR labels are placed at authorised rooms, parking areas, stores and maintenance areas.
- Individual employee accounts are created and assigned the correct role and department.

4.3.2 Taking an Asset

- The staff member logs in and opens the asset scan or asset action page.
- The asset QR code is scanned; the system loads the asset name, ID, current status, condition and latest location.
- The user scans or selects the destination/current hospital location.
- The user selects TAKE and may enter an optional reason or expected return information.
- The backend checks that the asset is available and the user is authorised.
- The database records the employee, asset, location, action and time.
- The asset changes from Available to In Use; the current holder is assigned and the available count decreases.

4.3.3 Returning an Asset

- The current holder scans or selects the asset.
- The correct return location or parking/store location is scanned or selected.
- The user selects RETURN.
- The backend verifies the holder and records the return transaction.
- The holder field is cleared, the returned time and location are stored, and the status changes to Available.
- Any open return request for that asset is completed and dashboard counts increase immediately.

4.3.4 Transferring an Asset

TRANSFER is used when an asset remains in use but moves to another authorised location or responsible employee. The new location and transfer details are recorded without incorrectly marking the asset as available, and the complete transfer remains visible in the transaction history for traceability.

4.3.5 Reporting Damage or Maintenance

- The user selects REPORT DAMAGE and enters the issue description.
- The system changes the asset to a maintenance-related state and removes it from the normal available quantity.
- Office users inspect the issue, update the maintenance stage and record completion.
- After repair and approval, the asset is restored to an authorised location and changed back to Available.

4.3.6 Management Monitoring

- Management opens the dashboard to view total registered assets and current status distribution.
- The user can identify available, in-use, return-requested, delayed, maintenance and missing assets.
- Recent movement records show the asset, employee, department, location, action and time.
- Reports and graphs help management identify high-use assets, shortages, delays and maintenance requirements.

4.4 Automatic Availability and Return Logic

Each asset type can store a minimum available quantity. The system compares the current available count with this minimum so that shortages can be recognised early.

State	Description
Normal availability	Available count is equal to or higher than the configured minimum.
Low availability	Available count falls below the configured minimum and the asset type is highlighted for attention.
Return request	The system or Office user contacts eligible holders, generally starting with assets taken earliest.
NEED response	The employee confirms that the asset is still required and enters the reason or expected return information.
RETURN response	The employee commits to return the asset within a selected time; the promised return time is stored.

Table 4.4 Availability and return-request states

Why this feature is intelligent

The system does not only store asset records. It compares actual availability with a configured minimum, identifies shortages and supports the recovery of assets that are already in use.

4.5 Messaging, Emergency and Maintenance

4.5.1 Internal Messages

The Asset Track system provides an internal messaging feature that allows authorised users to communicate operational information with employees, departments, or different user roles. Messages can be assigned different priority levels such as Normal, Urgent, or Emergency, depending on the importance of the information. Unread messages are highlighted through suitable indicators so that users can easily notice important updates while using the system. Each message is stored with details such as the sender, intended receiver or target group, priority level, message content, and time of communication, providing a clear record of internal communication.

4.5.2 Emergency Alerts

The emergency alert feature is designed to support quick communication during critical situations within the hospital. An authorised user can create an alert by selecting the emergency type, specifying the affected location, and entering a brief description of the situation. Once submitted, the alert is stored in the system and made available to the permitted user portals. The concerned users can acknowledge the alert using the response options provided by the system. After the situation has been brought under control, Office or Management users can close the alert. The completed emergency record remains stored in the system for future reference and review.

4.5.3 Maintenance Record

Asset Track maintains a dedicated maintenance history for each hospital asset. When a user reports damage, the issue is directly associated with the specific asset rather than being recorded as a separate complaint. While an asset is awaiting inspection or repair, the system prevents it from being counted as normally available. Information such as the reported issue, person who submitted the report, maintenance stage, and completion details is retained as part of the asset's maintenance history. This information enables Management to identify assets that require frequent repairs and supports decisions related to servicing, replacement, and future asset planning.

CHAPTER 5

SYSTEM DEVELOPMENT AND IMPLEMENTATION

5.1 Module-by-Module Development Process

The development of Asset Track was carried out in a systematic, module-by-module manner. Each part of the system was developed and checked before moving to the next stage. The process started with understanding the hospital's asset management requirements and continued through workflow design, database development, backend and frontend implementation, QR integration, security, testing, deployment, and documentation. This step-by-step approach helped ensure that the different modules worked together correctly and supported the required asset management operations.

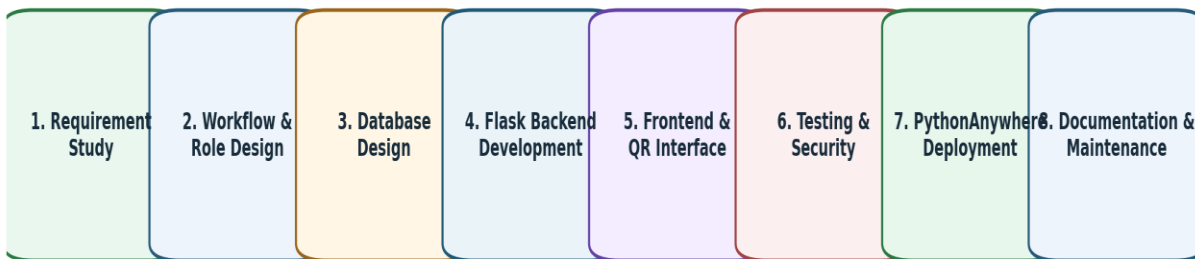


Figure 5.1. Module-by-module development process

5.1.1 Requirement Study

The first stage involved studying the hospital asset management problem and identifying the main requirements of the proposed system. The types of reusable assets, different user roles, required operations, reporting needs, and devices used for accessing the application were considered. Based on these requirements, the system was planned as a browser-based solution that could be accessed from commonly available devices. QR-based scanning was selected for asset and location identification, avoiding the need for costly hardware such as GPS or RFID equipment.

5.1.2 Workflow and Role Design

After identifying the requirements, separate workflows were prepared for **Nurse & Staff, Office, and Management** users. The responsibilities and permissions of each role were clearly defined to prevent unauthorised operations. Different asset states, such as Available, In Use, Maintenance, and other required conditions, were also mapped. These workflows helped the development team establish valid actions and prevent incorrect operations, such as taking an asset that is already assigned to another user.

5.1.3 Database Design

The database was designed using SQLite to organise and maintain the information required by Asset Track. Separate tables were created for users, departments, locations, asset categories, asset types, individual assets, transactions, return requests, maintenance records, messages, and emergency alerts. Unique IDs were used to connect related records and maintain proper relationships between different modules. The design also allows the system to store the current asset status while preserving previous transactions for future reference.

5.1.4 Flask Backend Development

The backend of the application was developed using Python and the Flask framework, with **app.py** serving as the main application file. Flask routes were created to handle requests received from the browser interface. The backend manages user sessions, verifies roles and permissions, validates asset conditions, performs database operations, and applies the required business rules. After processing each request, the appropriate information or HTML page is returned to the user.

5.1.5 Frontend Development

The frontend was developed using HTML, CSS, and JavaScript. HTML templates were used to create the different pages and dashboards, while CSS was applied to maintain a consistent and responsive layout. JavaScript was used for interactive functions such as QR and camera scanning. The interface was designed to work across different screen sizes, allowing staff to use mobile devices for asset operations while Office and Management users can use laptops or desktop computers for monitoring and administration.

5.1.6 QR and Camera Integration

QR codes were integrated into the system to provide a simple method of identifying assets and authorised locations. Each registered asset is associated with a unique QR value, and location QR codes are also created for approved areas. When a user scans a QR code using the device camera, the identifier is captured through JavaScript and sent to the Flask application. The system then retrieves the corresponding record and displays the relevant asset or location information.

5.1.7 Authentication and Permissions

Security features were implemented to ensure that system functions are available only to authorised users. Separate portal access and role-based permission checks were included for Staff, Office, and Management users. Passwords are securely handled using password hashing, while Flask sessions are used to maintain authenticated access. Logout functionality, account status checks, and password-reset support were also included to improve account security and usability.

5.1.8 Reports and Alerts

Once the main asset operations such as **Take, Return, and Transfer** were functioning correctly, additional monitoring and communication features were developed. These included asset status counts, recent transaction records, graphs, return-request handling, internal messages, emergency alerts, and maintenance screens. Alert and reporting functions allow users to identify important conditions such as low availability, delayed returns, maintenance requirements, and emergency situations.

5.1.9 Local Testing

Before deployment, the application was tested in the local development environment. The project was executed using the configured Python virtual environment and the available startup batch file. Camera-based QR scanning was also checked using a secure mobile-access method. Functional testing was performed for different user roles and major asset state changes to verify that the system responded correctly to valid and invalid operations.

5.1.10 Deployment

After completing the local development and testing stages, the application was prepared for cloud deployment using PythonAnywhere. The required Python packages were installed using **requirements.txt**, and the WSGI configuration was connected to the Flask application. The database, templates, and static files were also configured on the hosted environment. This made the Asset Track system accessible through a web browser using a secure HTTPS connection.

5.1.11 Documentation and Maintenance

The final stage involved preparing supporting documentation and maintenance resources for the project. **README.md** provides information about system setup and operation, while **UPDATE_NOTES.txt** can be used to record development changes and updates. A backup copy of the application is maintained through **app.zip**, and the password-reset utility supports account recovery when required. These resources help simplify future maintenance, troubleshooting, and further development of the Asset Track system.

Asset Track follows a straightforward design rule: the assets table always stores only the current state of each asset, while the transactions table stores the complete movement history. This keeps dashboards fast to query while preserving a full, traceable audit trail of every action.

Table	Important fields	Purpose
users	id, user_id, name, password hash, role, department_id, active	Login identity and role
departments	id, code, name	Hospital department master data
locations	id, code, name, department_id, qr_value	Rooms, stores and parking locations
asset_types	id, name, code, minimum_available, expected_return_minutes	Asset family rules
assets	id, asset_id, qr_value, status, condition, holder_id, location_id	Current state of each physical asset
transactions	asset_id, user_id, action, from_location, to_location, created_at	Permanent movement history
return_requests	asset_id, holder_id, response, promised_return_at, status	Low-availability return process
maintenance	asset_id, issue, reporter_id, stage, completed_at	Damage and repair history
messages	sender_id, target, priority, subject, body, seen	Internal communication
emergencies	type, location_id, description, reporter_id, status	Hospital-wide emergency records

Table 5.1 Main database tables and purpose

5.2 Database Design

5.2.1 Core SQLite schema (representative excerpt)

```
CREATE TABLE IF NOT EXISTS users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id TEXT UNIQUE NOT NULL,  
  employee_name TEXT NOT NULL,  
  password_hash TEXT NOT NULL,  
  role TEXT NOT NULL CHECK (role IN ('staff','office','management')),  
  department_id INTEGER,  
  is_active INTEGER NOT NULL DEFAULT 1,  
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE IF NOT EXISTS assets (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  asset_id TEXT UNIQUE NOT NULL,  
  asset_type_id INTEGER NOT NULL,  
  qr_value TEXT UNIQUE NOT NULL,  
  current_holder_id INTEGER,  
  current_location_id INTEGER,  
  status TEXT NOT NULL DEFAULT 'available',  
  condition_status TEXT NOT NULL DEFAULT 'good',  
  taken_at TEXT,  
  expected_return_at TEXT,  
  updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (current_holder_id) REFERENCES users(id),  
  FOREIGN KEY (current_location_id) REFERENCES locations(id)  
);  
  
CREATE TABLE IF NOT EXISTS transactions (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  asset_id INTEGER NOT NULL,  
  user_id INTEGER NOT NULL,  
  action TEXT NOT NULL,  
  from_location_id INTEGER,  
  to_location_id INTEGER,  
  notes TEXT,  
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (asset_id) REFERENCES assets(id),  
  FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

Database rule

The assets table stores only the current state. The transactions table stores the complete history. Therefore, dashboards are fast to load and every previous movement remains traceable.

5.3 Project Folder and File Structure

Name	Status	Date modified	Type	Size
.env	✓	16-07-2026 21:04	File folder	
static	✓	16-07-2026 21:04	File folder	
templates	✓	16-07-2026 21:04	File folder	
app.py	✓	16-07-2026 21:04	Python File	48 KB
app.zip	✓	26-07-2026 13:46	Compressed (zipp...	68 KB
meditrack.db	✓	26-07-2026 07:46	Data Base File	112 KB
Procfile	✓	16-07-2026 21:04	File	1 KB
README.md	✓	16-07-2026 21:04	MD File	3 KB
render.yaml	✓	16-07-2026 21:04	YAML File	1 KB
requirements.txt	✓	16-07-2026 21:04	Text Document	1 KB
reset_password.py	✓	26-07-2026 07:03	Python File	3 KB
START_MEDTRACK.bat	✓	16-07-2026 21:04	Windows Batch File	1 KB
START_SECURE_MOBILE_LINK.bat	✓	16-07-2026 21:04	Windows Batch File	1 KB
UPDATE_NOTES.txt	✓	16-07-2026 21:04	Text Document	1 KB

Figure 5.3 Completed project folder used for local development and deployment

File / Folder	Type	Purpose
.env	Folder	Local Python virtual environment containing isolated project packages
static	Folder	CSS, JavaScript, icons, images and other browser assets
templates	Folder	Flask HTML pages for home, login, dashboards, scan screens and reports
app.py	Python	Main Flask application: routes, authentication, business logic and database access
app.zip	Archive	Backup or transferable copy of the application

File / Folder	Type	Purpose
meditrack.db	SQLite database	Stores the complete operational data of the system
Procfile	Deployment file	Start-command configuration for Render/Heroku-style hosting
render.yaml	YAML	Alternative Render deployment configuration retained in the project
requirements.txt	Text	List of Python libraries required to run the application
README.md	Documentation	Project setup, run and usage instructions
reset_password.py	Python utility	Administrative password recovery/reset support
START_MEDTRACK.bat	Windows batch	Starts the local Flask application quickly on Windows
START_SECURE_MOBILE_LINK.bat	Windows batch	Starts a secure mobile-access link for camera/QR testing
UPDATE_NOTES.txt	Text	Records modifications, fixes and version notes

Table 5.2 Project folder and file explanation

5.4 Request Flow and User Interface

Every user action in Asset Track – a click, a form submission or a QR scan – follows the same coding path from the browser through to the database and back, which keeps the system predictable and easy to debug.

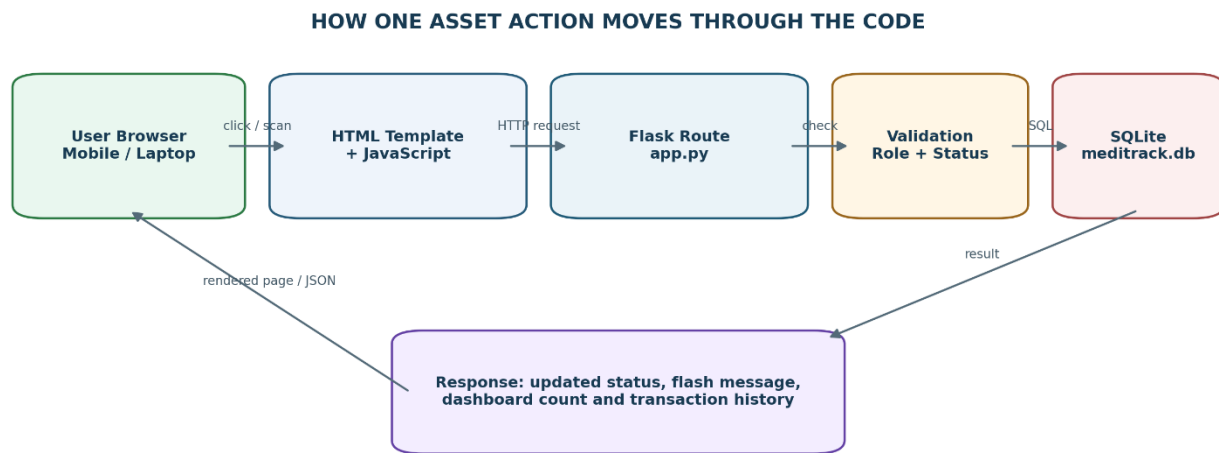


Figure 5.4 How one asset action moves through the code, from browser to database and back

- The user submits a form or scans a QR code in an HTML page.
- JavaScript validates the basic input and sends the request to a Flask route.
- The Flask route checks the session, portal role, asset status and required fields.
- SQL reads or updates meditrack.db inside a controlled transaction.
- The route records the action history and returns a page redirect or JSON response.
- The browser shows the new status and the dashboards read the updated counts.

5.4.1 Main Interface Screens

- Home page – presents the Asset Track name and three portal buttons (Nurse & Staff, Office, Management) leading to the matching login form.
- Login page – collects the User ID and password for the selected portal and shows a clear error when the credentials or role do not match.
- Staff scan / action page – activates the device camera, decodes the asset QR code, displays the asset's current status and location, and presents Take / Return / Transfer / Report Damage buttons appropriate to that status.
- Staff dashboard – lists assets currently assigned to the logged-in employee and any pending return requests awaiting a NEED / RETURN response.

- Office dashboard – shows master-data management (employees, departments, locations, asset types, assets), current holders/locations, low-availability warnings and maintenance queues.
- Management dashboard – presents status totals, graphs, the most recent transactions, delayed/maintenance/missing asset lists and downloadable reports.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Testing and Demonstration Procedure

The completed system was tested functionally for every user role and every asset state transition, using at least two separate accounts and browser sessions to confirm that concurrent actions are handled safely.

Test	Action	Expected result
Portal login	Login using each role through its correct portal	Correct dashboard opens; wrong role is rejected
Take asset	Scan/select an available asset and choose TAKE	Status becomes In Use and holder/location/time are recorded
Duplicate take	Try to take an asset already in use	System blocks the invalid second assignment
Return asset	Return the same asset at an authorised location	Holder clears, status becomes Available and count increases
Transfer	Move an in-use asset to another location	Location/history update while asset remains In Use
Damage report	Report a fault for an available or in-use asset	Asset changes to maintenance-related state and issue is recorded
Low availability	Reduce available quantity below the minimum	Low-availability warning/return process appears
Return promise	Respond RETURN with a time, then allow it to pass	Return Requested changes to Delayed if not returned
Messages	Send a message from one role/device to another	Receiver sees the correct message and priority
Emergency	Create and acknowledge an emergency alert	Permitted portals receive it and acknowledgements are stored
Reports	Open dashboard/history and generate a report/export	Counts and records match the performed transactions
Responsive/camera	Test on phone, tablet and laptop	Pages remain usable and camera works through HTTPS permission

Table 6.1 Functional test cases and expected results

6.1.1 Demonstration Scenario

- Office selects a wheelchair record, for example WC-004, and confirms its QR code and default parking location.
- A Staff user scans WC-004, selects ICU 1 and chooses TAKE; Office confirms that it is In Use under the correct employee.
- Management checks the available count and recent transaction; the Staff user then demonstrates a transfer or return-request response.
- The Staff user returns WC-004 to Wheelchair Parking, so the status becomes Available and the full history remains stored.
- Another demo asset is used to show damage/maintenance, followed by a message and an emergency acknowledgement.

6.1.2 Asset Track Portal Selection Page

The Asset Track Portal Selection Page is the initial interface through which users access the hospital asset tracking system. The page provides three separate portal options: Nurse & Staff, Office, and Management. Each portal is designed according to the responsibilities and access requirements of the respective user group.

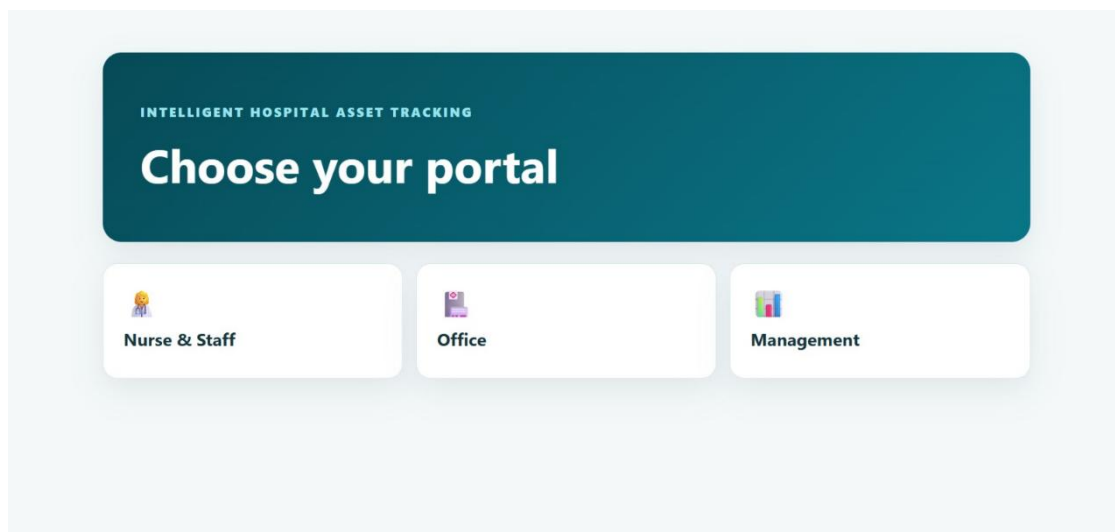


Figure 6.1.2 Asset Track Portal Selection Page

6.1.3 Nurse & Staff Login Page

The **Nurse & Staff Login Page** provides a secure authentication interface for authorised hospital staff. The page contains separate fields for entering the **User ID** and **Password**, followed by a Login button. Before accessing the system, the user must provide valid authentication details.

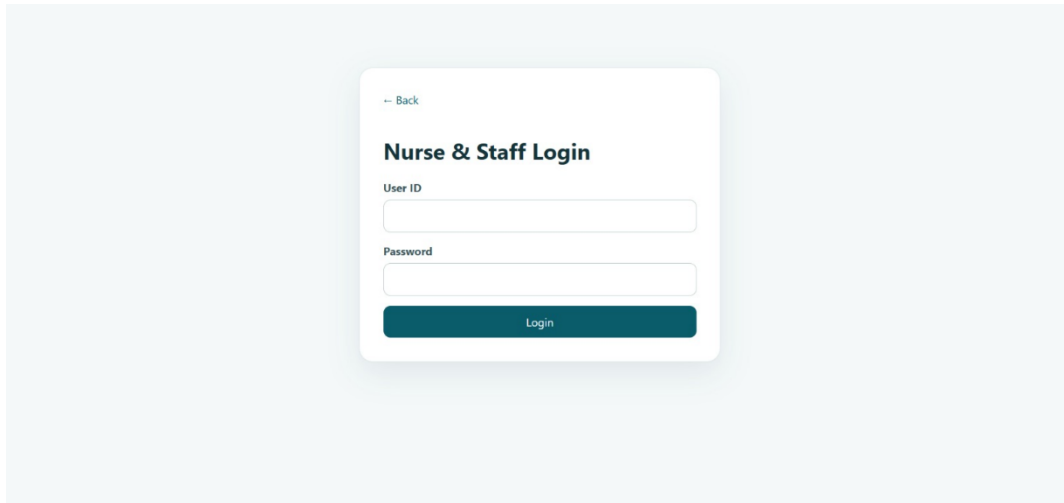


Figure 6.1.3. Nurse & Staff Login Page

6.1.4 Staff Dashboard

The Staff Dashboard provides an overview of the user's activities and assigned assets. It includes options for asset QR scanning, messages, emergency access, and logout. The dashboard also displays currently assigned assets and recent notifications.

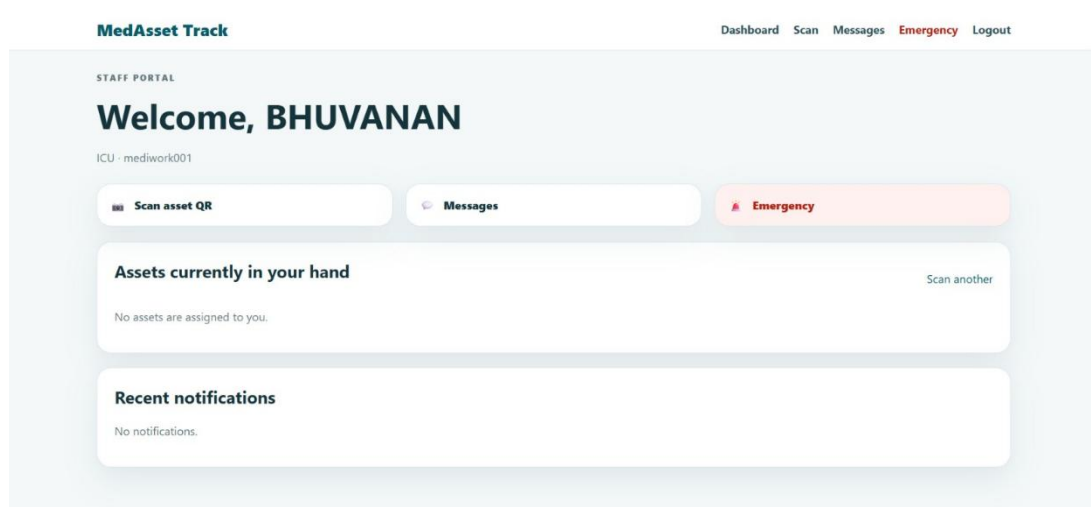


Figure 6.1.4 – Staff Dashboard

6.1.5 Asset QR Code Scanning

The QR tracking page allows staff to identify hospital assets by scanning their QR codes. A manual asset ID or QR value can also be entered when camera-based scanning is unavailable. This enables quick and efficient asset identification.

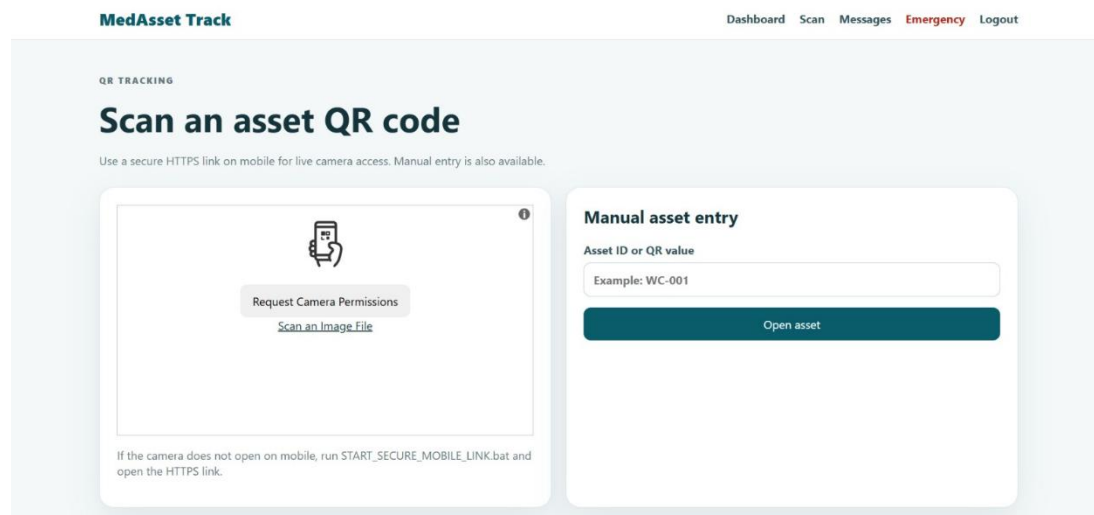


Figure 6.1.5 Asset QR Code Scanning

6.1.6 Current Asset Overview

The Current Asset Overview provides a real-time view of hospital assets. It displays the number of assets that are available, in use, under maintenance, or delayed. The system also categorizes assets such as infusion equipment, patient monitoring equipment, patient transport equipment, and respiratory equipment.

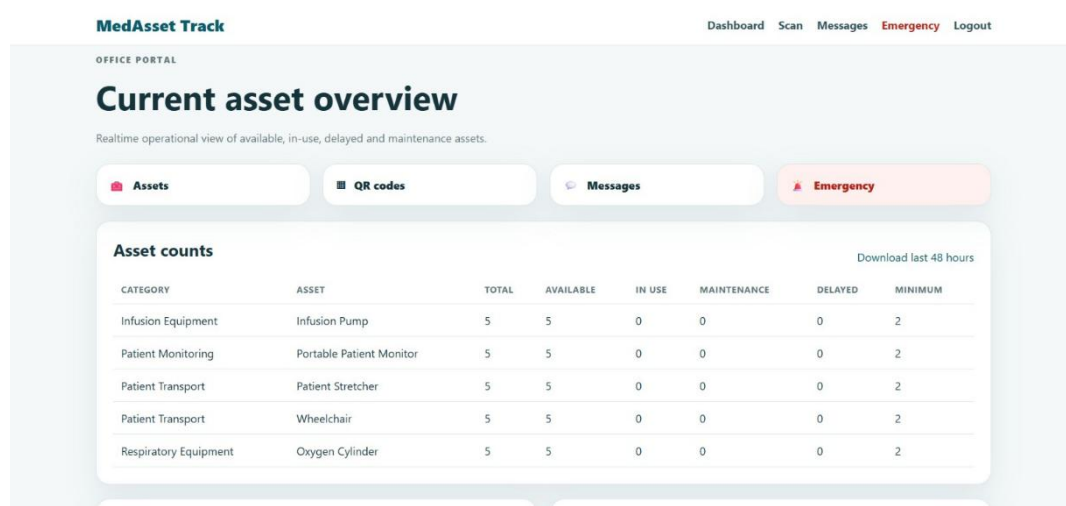
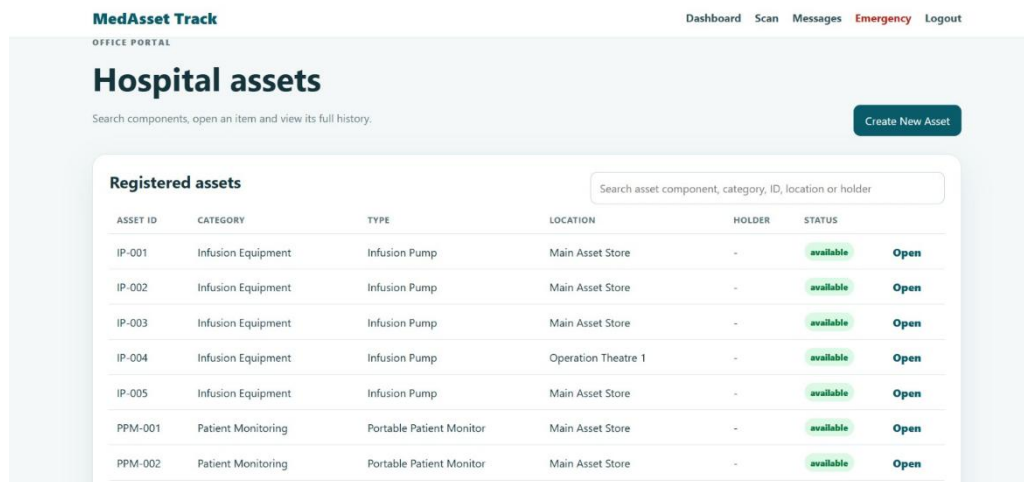


Figure 6.1.6 Current Asset Overview

6.1.7 Hospital Assets

The Hospital Assets page maintains a centralized list of registered assets. Each asset is provided with an Asset ID, category, type, location, holder, and status. The search option helps users quickly locate specific assets, while the system also provides an option to create a new asset.



MedAsset Track OFFICE PORTAL Dashboard Scan Messages Emergency Logout

Hospital assets

Search components, open an item and view its full history. [Create New Asset](#)

Registered assets

ASSET ID	CATEGORY	TYPE	LOCATION	HOLDER	STATUS	
IP-001	Infusion Equipment	Infusion Pump	Main Asset Store	-	available	Open
IP-002	Infusion Equipment	Infusion Pump	Main Asset Store	-	available	Open
IP-003	Infusion Equipment	Infusion Pump	Main Asset Store	-	available	Open
IP-004	Infusion Equipment	Infusion Pump	Operation Theatre 1	-	available	Open
IP-005	Infusion Equipment	Infusion Pump	Main Asset Store	-	available	Open
PPM-001	Patient Monitoring	Portable Patient Monitor	Main Asset Store	-	available	Open
PPM-002	Patient Monitoring	Portable Patient Monitor	Main Asset Store	-	available	Open

Figure 6.1.7 Hospital Assets

6.1.8 Asset QR Codes

The Asset QR Codes page displays unique QR codes assigned to individual hospital assets. Each QR code is associated with a specific Asset ID, enabling quick identification and tracking of equipment within the hospital.

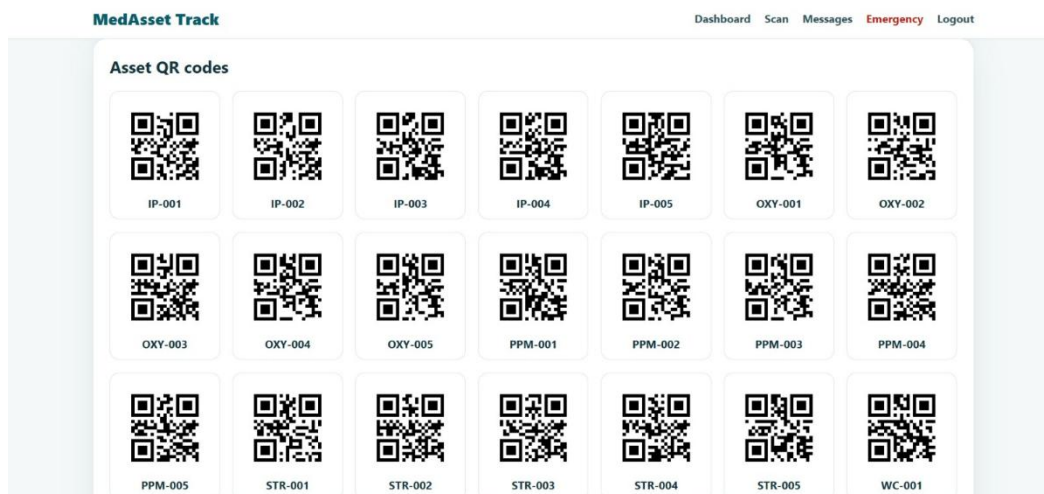


Figure 6.1.8 Asset QR Codes

6.1.9 Create New Asset

The Create New Asset page allows authorized users to register new hospital assets. Details such as asset category, asset name, asset code prefix, QR/asset ID, default location, minimum alert count, and return time can be entered. A QR code is generated automatically for the newly created asset.

Create New Asset

Enter the asset category, asset name, asset ID and location. A QR code is generated automatically.

Asset category	Asset name	Asset code prefix	QR / asset ID	Default location
Example: Patient Transport	Example: Wheelchair	Example: WC	Example: WC-006	Ambulance Parking Bay ▾

Minimum alert count	Return time (minutes)
2	10

Create asset

Figure 6.1.9 Create New Asset

6.1.10 Send Message

The messaging page provides communication between users and hospital staff. Users can select the recipient, priority, subject, and message content before sending. The inbox section allows received messages to be viewed.

MedAsset Track

Dashboard Scan Messages **Emergency** Logout

Send message

Send to role
Individual user ▾

Individual user
None ▾

Priority
Normal ▾

Subject

Message

Send message

Inbox

No messages.

Figure 6.1.10 Send Message

6.1.11 Hospital Asset Analytics

The Hospital Asset Analytics dashboard provides an overall view of asset utilization and operational status. It displays total assets, available assets, assets in use, maintenance, delayed, and missing assets. Graphs provide a visual representation of assets by category and recent transactions, supporting better monitoring and decision-making. □

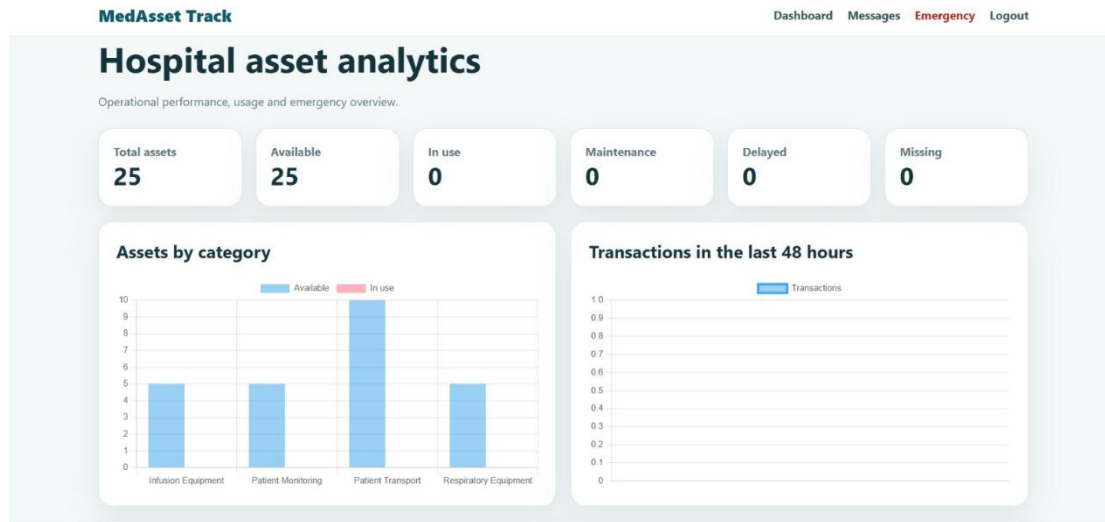


Figure 6.1.11 Hospital Asset Analytics

6.2 Results and Discussion

The completed Asset Track application successfully demonstrated end-to-end identification, assignment and recovery of hospital assets using only QR-code scanning and a standard web browser. Every functional test listed in Table 7.1 produced the expected result, confirming that role-based access control, asset status validation and transaction logging behave correctly under normal use and under attempted invalid actions such as taking an already in-use asset.

The Take → In Use → Return → Available cycle, together with the Transfer and Damage/Maintenance exception paths, correctly updated the current holder, location and status for every tested asset, and the corresponding entry was always visible in the transaction history reviewed from the Office and Management dashboards. The automatic availability logic correctly flagged asset types as low-availability once their available count fell below the configured minimum, and delayed-return detection correctly changed status from Return Requested to Delayed once a promised return time passed without action.

Messaging and emergency alert features were confirmed to reach the intended portals with the correct priority level, and acknowledgements were correctly stored against the emergency record. Camera-based QR scanning worked reliably on mobile devices once HTTPS and camera permission were granted, and a manual code-entry fallback remained available for situations where scanning was not possible.

Overall, the results confirm that a browser-based, QR-driven approach can reliably replace manual, notebook-based asset tracking in a hospital setting. The system gives Nurse & Staff, Office and Management users a single, consistent and traceable view of asset availability, reduces the time required to locate shared equipment, and exposes shortages and delays before they affect patient care – achieving the objectives set out in Chapter 1 without the cost of RFID, BLE or GPS-based hardware infrastructure.

CHAPTER 7

SCOPE OF FUTURE WORK

Advantages	Current limitations	Future improvements
Low-cost, browser-based; no dedicated tracking hardware; complete auditable history; role-based access; automatic shortage/delay alerts.	Shows the latest confirmed location rather than continuous live movement; camera scanning needs HTTPS; SQLite needs planning for very large deployments.	RFID/BLE positioning; native mobile app; PostgreSQL/MySQL migration; predictive maintenance; HIS integration.

Table 7.1 Advantages, limitations and future improvements

Beyond the improvements summarised above, the following specific directions can extend the system further:

- RFID, BLE or UWB-based positioning can be added for continuous real-time location instead of latest-scan confirmation.
- Push notifications or a native mobile app can replace browser-only alerts for faster staff response.
- The database can be migrated from SQLite to PostgreSQL or MySQL to support a larger number of concurrent users and assets.
- Predictive maintenance scheduling can be added using the accumulated maintenance history of each asset type.
- Analytics and usage-pattern dashboards can help forecast asset procurement needs across departments.
- Integration with an existing Hospital Information System (HIS) can unify asset records with patient-care workflows.
- Audit-log export and automated cloud backup can strengthen compliance and disaster recovery.

7.1 Advantages

- Low-cost browser-based system; no specialised tracking hardware required.
- Role-based access keeps Staff, Office and Management functions clearly separated.
- Complete, tamper-evident movement history for every asset.

- Faster asset search and reduced idle staff time.
- Automatic shortage and delayed-return warnings.
- Linked maintenance and management reporting

7.2 Limitations

- Shows the latest confirmed location rather than continuous live movement.
- Camera-based scanning requires HTTPS and browser camera permission.
- Browser-based alerts may stop when the device browser is closed.
- SQLite requires regular backup and careful planning before scaling to very large deployments.

7.3 Applications

- General hospitals and multi-speciality medical centres.
- Diagnostic centres and standalone clinics with shared portable equipment.
- Ambulance services tracking transportable emergency equipment.
- Educational and training hospitals for equipment accountability.
- Any institution requiring low-cost, auditable tracking of reusable movable assets.

CHAPTER 8

APPENDICES

Appendix – 1 Login Credentials

The following demonstration accounts were configured for evaluation of the live Asset Track deployment. In real hospital use, shared demonstration accounts must be replaced with individual employee accounts as noted in Section 4.2.

Live website: <https://assettrack.pythonanywhere.com>

Role / Access Level	Username	Password
Nurse & Staff	medwork001	12345678
Nurse & Staff	medwork002	12345678
Office Staff	office123	12345678
Management	management123	12345678

Appendix – 2 Database Schema Specification

Complete field-level specification of the core SQLite tables used by Asset Track.

Table	Key fields	Notes
users	id (PK), user_id (unique), employee_name, password_hash, role, department_id, is_active, created_at	role restricted to staff / office / management
departments	id (PK), code, name	Master data created by Office
locations	id (PK), code, name, department_id (FK), qr_value (unique)	Includes rooms, stores and parking points
asset_types	id (PK), name, code, minimum_available, expected_return_minutes	Defines shortage threshold and expected return time
assets	id (PK), asset_id (unique), asset_type_id (FK), qr_value (unique), current_holder_id (FK), current_location_id (FK), status, condition_status, taken_at, expected_return_at, updated_at	Stores only the current state of each asset

Table	Key fields	Notes
transactions	id (PK), asset_id (FK), user_id (FK), action, from_location_id, to_location_id, notes, created_at	Append-only movement history
return_requests	id (PK), asset_id (FK), holder_id (FK), response, promised_return_at, status	Drives the low-availability return workflow
maintenance	id (PK), asset_id (FK), issue, reporter_id (FK), stage, completed_at	Damage and repair history
messages	id (PK), sender_id (FK), target, priority, subject, body, seen, created_at	Internal communication log
emergencies	id (PK), type, location_id (FK), description, reporter_id (FK), status	Hospital-wide emergency record

Appendix – 3 Pseudo Code for Key Functions

The following structured pseudocode summarises the core Take-Asset operation implemented in the Flask backend (app.py), simplified from the actual source code for presentation purposes.

A3.1 Take Asset Operation

```

BEGIN Take_Asset(asset_id, location_id, user)
  IF asset_id or location_id is missing THEN
    SHOW "Asset and location are required"; RETURN
  END IF

  asset ← SELECT asset record FROM assets WHERE id = asset_id
  IF asset NOT FOUND THEN
    SHOW "Asset not found"; RETURN
  END IF

  IF asset.status ≠ 'available' THEN
    SHOW "This asset is not currently available"; RETURN
  END IF
  IF asset.condition NOT IN {'good', 'minor_damage'} THEN
    SHOW "This asset cannot be issued in its current condition"; RETURN
  END IF

  expected_return ← now() + asset_type.expected_return_minutes

  BEGIN TRANSACTION
    UPDATE assets SET status='in_use', holder=user,
      location=location_id, taken_at=now(),
      expected_return_at=expected_return
      WHERE id = asset_id AND status = 'available'
    INSERT INTO transactions(asset_id, user, action='take',
      from_location, to_location, time=now())

```

```
COMMIT TRANSACTION

SHOW "Asset issued successfully"
END Take_Asset
```

A3.2 Automatic Low-Availability and Delay Check

```
BEGIN Check_Availability_And_Delays()
  FOR EACH asset_type IN asset_types DO
    available_count ← COUNT(assets WHERE type = asset_type
                           AND status = 'available')
    IF available_count < asset_type.minimum_available THEN
      FLAG asset_type AS 'low_availability'
      CREATE return_request FOR earliest-taken asset(s)
    END IF
  END FOR

  FOR EACH asset IN assets WHERE status IN
    ('in_use','return_requested') DO
    IF asset.expected_return_at < now() THEN
      SET asset.status = 'delayed'
    END IF
  END FOR
END Check_Availability_And_Delays
```

A3.3 Role-Based Access Check (applied to every protected route)

```
BEGIN Roles_Required(allowed_roles)
  IF current_user is NOT logged in THEN
    REDIRECT to home page; RETURN
  END IF
  IF current_user.role NOT IN allowed_roles THEN
    RETURN HTTP 403 Forbidden
  END IF
  PROCEED to requested page
END Roles_Required
```

Appendix – 4 System Development and Coding Checklist

Area	Checklist
Configuration	Production SECRET_KEY is not written directly in app.py
Passwords	All passwords are stored using generate_password_hash
Queries	All user-supplied values use SQL ? parameters (no string concatenation)
Sessions	Every protected page checks login and role before rendering
Asset rules	Take, Return, Transfer and Damage validate the current status before writing
Transactions	Every successful movement creates a transaction history record
Database writes	Multi-step actions commit together or roll back together
Camera	HTTPS and a manual-code fallback are provided for QR scanning
Errors	Users receive clear messages without exposing database internals

CHAPTER 9

CONCLUSION

This project successfully demonstrated the design, development and deployment of Asset Track, an Intelligent Hospital Asset Tracking and Management System built using a Python Flask backend, an SQLite database and a QR-code-driven browser interface. The main objective was to give a hospital a low-cost, common system to identify reusable assets, record who is using each asset, confirm its latest location, control availability, support timely returns, report damage and provide clear management reports – without relying on expensive RFID, BLE or GPS tracking hardware. Separate role-based portals for Nurse & Staff, Office and Management ensured that each user group could access only the functions relevant to their responsibilities, while every take, return, transfer and damage action was recorded as a permanent, auditable transaction. The automatic low-availability and delayed-return logic proved effective in surfacing shortages and overdue assets before they could disrupt hospital operations, and the messaging and emergency-alert modules extended the system beyond simple tracking into a broader operational communication tool. Functional testing across all user roles and asset states confirmed that the system behaves correctly under both normal use and attempted invalid actions, and that the responsive interface performs reliably across mobile, tablet and desktop devices. The live HTTPS deployment on PythonAnywhere demonstrated that the complete solution can be hosted and operated at low recurring cost, making it a practical option for hospitals, clinics and other institutions that need dependable, low-cost tracking of shared, reusable equipment. Overall, this project confirms that a software-only, QR-based approach, when combined with careful workflow design, role-based access control and automatic availability logic, can deliver most of the operational benefit of costlier hardware-based asset tracking systems, while remaining affordable and easy to deploy for resource-constrained healthcare institutions. Future enhancements such as RFID/BLE positioning, native mobile applications, predictive maintenance and HIS integration, discussed in Chapter 8, provide a clear roadmap for extending the system's capability.

REFERENCES

- [1] Zhu, X., Tao, Y., Zhu, R., Wu, D., & Ming, W. K. (2022). *Impact of Hospital Characteristics and Governance Structure on the Adoption of Tracking Technologies for Clinical and Supply Chain Use: Longitudinal Study of US Hospitals*. Journal of Medical Internet Research, 24(5), e33742. <https://www.jmir.org/2022/5/e33742/>

- [2] Mao, L., Zheng, K., Shen, Y., & Lin, Z. (2022). *Whole Life Cycle Asset Tracking Management System of Medical Equipment Based on RFID*. China Medical Devices, 37(1), 9–11. DOI:10.3969/j.issn.16741633.2022.01.003. <https://doi.org/10.3969/j.issn.16741633.2022.01.003>

- [3] *Understanding the RFID Deployment at Sacred Heart Medical Center: Using Technology-Organization-Environment Framework Lenses*. (2022). Procedia Computer Science, 196, 445–453. DOI: 10.1016/j.procs.2021.12.035. <https://doi.org/10.1016/j.procs.2021.12.035>

- [4] Han, Wang, Chu, & Jiao. (2022). *A New Model of Intelligent Nursing Based on 5G Network Architecture and RFID Technology*. Mobile Information Systems. Article ID 6586213. <https://doi.org/10.1155/2022/6586213>

- [5] Azzahra, Z. F., & Suakanto, S. (2025). *A Barcode-Based Medical Asset Management System with Integrated Monitoring in Hospitals*. JUSIFO (Jurnal Sistem Informasi), 11(1), 21–30. DOI: 10.19109/jusifo.v11i1.25268. <https://doi.org/10.19109/jusifo.v11i1.25268>

- [6] Stanimirović, D., Umek, L., & Ravšelj, D. (2025). *Exploring Research Trends in Healthcare Asset Management Systems: Key Areas, Strategic Priorities, and Actionable Recommendations*. Medicine, Law & Society, 18(2), 333–358. DOI: 10.18690/mls.18.2.333-358.2025. <https://doi.org/10.18690/mls.18.2.333-358.2025>

- [7] Suakanto, S., Nuryatno, E. T., Fakhurroja, H., & Rijadi, S. C. R. (2026). *Enhancing Healthcare RFID Asset Tracking: A Multi-Objective Optimization Approach Considering Network Delay, False Identification, and Energy Efficiency*. Journal of ICT Research and Applications, 19(3). DOI: 10.5614/itbj.ict.res.appl.2026.19.3.1. <https://doi.org/10.5614/itbj.ict.res.appl.2026.19.3.1>

- [8] Yu, W., Chong, K. L., & Lamsali, H. B. (2026). *IoT-enabled Lifecycle Management of Medical Equipment Assets in Smart Hospitals: A Framework and Case Study*. African Journal of Science, Technology, Innovation and Development. DOI: 10.1080/20421338.2026.2681950. <https://doi.org/10.1080/20421338.2026.2681950>

- [9] Blanck et al. (2026). *BLE Tracking Systems for Optimization of Pharmacy Crash Cart Operations*. American Journal of Health-System Pharmacy, 83(15), e588–e594. DOI: 10.1093/ajhp/zxag046.
<https://doi.org/10.1093/ajhp/zxag046>
- [10] Joshi, P., & Sawant, S. (2024). *The Impact and Potential of Quick Response (QR) Codes in Healthcare: A Comprehensive Review*. Proceedings of the Human Factors and Ergonomics Society Annual Meeting, 68(1). DOI: 10.1177/10711813241278266.
<https://doi.org/10.1177/10711813241278266>
- [11] Yao, W., Chu, C.-H., & Li, Z. (2012). *The Adoption and Implementation of RFID Technologies in Healthcare: A Literature Review*. Journal of Medical Systems, 36(6), 3507–3525.<https://pubmed.ncbi.nlm.nih.gov/22009254/>
- [12] Fisher, J. A., & Monahan, T. (2008). *Tracking the Social Dimensions of RFID Systems in Hospitals*. International Journal of Medical Informatics, 77(3), 176–183. DOI: 10.1016/j.ijmedinf.2007.05.005.
<https://doi.org/10.1016/j.ijmedinf.2007.05.005>
- [13] Ting, S. L., Kwok, S. K., Tsang, A. H. C., & Lee, W. B. (2011). *Critical Elements and Lessons Learnt from the Implementation of an RFID-enabled Healthcare Management System in a Medical Organization*. Journal of Medical Systems, 35(4), 657–669.
<https://ira.lib.polyu.edu.hk/handle/10397/4807>
- [14] Tajima, M. (2007). *Strategic Value of RFID in Supply Chain Management*. Journal of Purchasing and Supply Management, 13(4), 261–273. DOI: 10.1016/j.pursup.2007.11.001.
<https://doi.org/10.1016/j.pursup.2007.11.001>
- [15] Wang, S. W., Chen, W. H., Ong, C. S., Liu, L., & Chuang, Y. W. (2006). *RFID Applications in Hospitals: A Case Study on a Demonstration RFID Project in a Taiwan Hospital*. Proceedings of the 39th Annual Hawaii International Conference on System Sciences.<https://scholars.lib.ntu.edu.tw/handle/123456789/472575>
- [16] Ngai, E. W. T., Moon, K. K. L., Riggins, F. J., & Yi, C. Y. (2008). *RFID Research: An Academic Literature Review (1995–2005) and Future Research Directions*. International Journal of Production Economics, 112(2), 510–520.<https://doi.org/10.1016/j.ijpe.2007.05.004>
- [17] Shokouhifar, M. (2022). *Swarm Intelligence RFID Network Planning Using Multi-Antenna Readers for Asset Tracking in Hospital Environments*. Computer Networks. DOI: 10.1016/j.comnet.2021.108427.<https://doi.org/10.1016/j.comnet.2021.108427>

- [18] Chowdhury, B., & Khosla, R. (2007). *RFID-based Hospital Real-time Patient Management System*. Proceedings of the 6th International Conference on Computer Information Systems and Industrial Management Applications, 363–368. <https://3icare.com/files/Chowdhury-RFIDHPMS.pdf>
- [19] Saritha, K., Cherukuri, A., Poreddy, A. R., Sundralla, S. S., Nayab, R. S., Alzubaidi, L. H., Vijilius, H. R., Dutt, A., & Yadav, D. K. (2024). *IoT Enabled Hospital Asset Tracking Using Advanced Interdisciplinary Approaches*. E3S Web of Conferences, 507, 01007. <https://doi.org/10.1051/e3sconf/202450701007>
- [20] Landt, J. (2005). *The History of RFID*. IEEE Potentials, 24(4), 8–11. <https://ieeexplore.ieee.org/document/1549751/>